

×

^

- Lesson 1.4 Summary and Assessments
- Conclusion
Video • 3 min
 - Check your understanding: End of module 1
Graded Assignment • Grade: 100%

Course Syllabus



Course Description

Target Audience

- Computer science students seeking to strengthen their theoretical and practical understanding of algorithms
- Software developers who want to improve their algorithm design and analysis skills
- Technical professionals who need to evaluate and optimize computational solutions
- Data scientists working with large datasets requiring efficient processing
- Anyone interested in the fundamental limits and capabilities of computational systems

Content Differentiation

- Connects abstract computational models (automata) to concrete implementation techniques
- Provides hands-on experience with algorithm design alongside formal analysis
- Develops both mathematical rigor and practical intuition for algorithm performance
- Emphasizes the reasoning process behind algorithm selection and optimization
- Builds a bridge between theoretical computer science and software engineering practice

Course Goals and Objectives

- Design and analyze deterministic and non-deterministic finite automata
- Implement and compare various searching and sorting algorithms
- Evaluate algorithm efficiency using asymptotic notation and complexity analysis
- Develop and optimize recursive and iterative algorithm implementations
- Apply the master theorem to analyze divide-and-conquer algorithms
- Classify computational problems by complexity class
- Reason about algorithm correctness and performance trade-offs
- Recognize the practical implications of theoretical complexity limits

The course consists of four modules that progressively build your understanding of algorithms and complexity:

Module 2: Searching and Sorting Algorithms Examines fundamental algorithmic techniques for organizing and retrieving data, introducing key concepts in algorithm design, representation, and analysis.

Module 2: Searching and Sorting Algorithms Examines fundamental algorithmic techniques for organizing and retrieving data, introducing key concepts in algorithm design, representation, and analysis.

Module 4: Complexity Theory Investigates the efficiency and scalability of algorithms through asymptotic analysis, Big O notation, and complexity classes, providing tools to reason about computational limits.

Module 4: Complexity Theory Investigates the efficiency and scalability of algorithms through asymptotic analysis, Big O notation, and complexity classes, providing tools to reason about computational limits.

Module 1: Automata Theory

Module 1: Automata Theory

- Define the basic terminologies of automata theory
- Describe finite automata and what they can represent
- Build deterministic and nondeterministic finite automata
- Apply concepts in automata theory, such as deterministic automata and regular languages
- Recognize the languages that different automata can accept
- Use automata simulators to visualize and test theoretical concepts

Module 2: Searching and Sorting Algorithms

By the end of this module, you will:

- Specify the process of algorithmic thinking and apply proof techniques
- Implement various sorting algorithms including insertion sort, bubble sort, and selection sort
- Compare algorithm performance in different contexts
- Apply binary search and understand its efficiency advantages
- Represent algorithms using pseudocode and flowcharts
- Analyze the best, average, and worst-case performance of algorithms
- Implement heap structures and heapsort

Module 3: Recursive and Iterative Algorithms

By the end of this module, you will:

- Distinguish between recursive and iterative algorithm implementations
- Design and analyze divide-and-conquer algorithms like quicksort
- Implement merge algorithms and merge sort
- Apply the Gale-Shapley algorithm to solve matching problems
- Analyze stable matching and allocation problems
- Convert between recursive and iterative algorithm implementations
- Identify scenarios where recursion or iteration is more appropriate

Module 4: Complexity Theory

By the end of this module, you will:

- Analyze algorithm efficiency using asymptotic notation
- Apply Big O notation to characterize algorithm performance
- Calculate complexity for recursive algorithms using the master theorem
- Compare the efficiency of common sorting algorithms
- Understand the concept of best, average, and worst-case complexity
- Recognize the significance of complexity classes like P and NP
- Apply complexity analysis to practical algorithm selection

✓ Completed